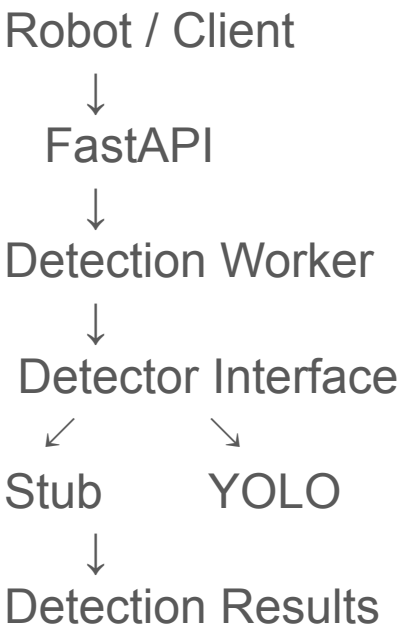
Week 4 Progress Report

InGen Robotics Perception Backend

- Built a FastAPI backend for robot perception workflows
- Designed versioned `/v1/` API endpoints for frame submission and retrieval
- Used Pydantic v2 for request/response validation
- Designed asynchronous inference architecture
- Prepared the system for YOLO-based object detection



Versioned API and Data Contracts

Implemented:

POST /v1/frames

GET /v1/frames/{frame_id}

GET /v1/robots/{robot_id}/state

GET /v1/detections

```
{
  "robot_id": "aido_001",
  "platform": "aido",
  "timestamp": "...",
  "image_base64": "..."
}
```

Key features:

- Explicit Pydantic request/response models
- JPEG + Base64 frame ingestion
- Image validation with Pillow
- UTC timestamp validation
- Normalized bounding boxes
- Structured error responses

Decoupled Inference Pipeline

Created a common Detector interface:

load()

warm_up()

predict(image)

unload()

Implemented:

- `StubDetector`
  - deterministic results
  - used in CI/testing
  - no GPU or model weights required
- `YoloDetector`
  - Ultralytics YOLO
  - converts predictions into structured `Detection` objects

DetectionWorker responsibilities:

- Load model once
- Warm model before serving traffic
- Manage bounded queue
- Track `ready` / `healthy`
- Enforce inference timeout
- Gracefully shut down

Reliability & Code Quality

Testing

- pytest
- async worker tests
- API tests
- model validation tests
- queue/full/timeout/shutdown behavior

Code quality

- Black
- Ruff
- mypy
- pytest-cov

CI

- GitHub Actions automatically runs checks on pushes

Results

- 20+ tests passing
- ~87–90% line coverage during development
- structured logging
- request-ID correlation

Engineering lessons

- Avoid duplicate shared worker instances
- Keep API and inference responsibilities separate
- Use bounded queues to prevent overload
- Use stub implementations to keep CI fast and deterministic